

第二大脑 LLM-Wiki 让 AI 替你维护知识库

Karpathy 的玩法：Obsidian 是 IDE，LLM 是程序员，wiki 是代码库

"The wiki is a persistent, compounding artifact."

——wiki 是一个持久的、会复利的资产 (Andrej Karpathy 《llm-wiki》gist, 5000+ star)

一手资料：karpathy/llm-wiki gist · 社区实现：Aquinas-Protocol/second-brain · 2026-10-02 · F:\reminder
拾遗队列

1 第二大脑是什么？

2 和 RAG 区别在哪？

3 为什么人的 wiki 会烂？

4 三层架构怎么分权？

5 原子笔记与双链？

6 Ingest/Query/Lint 循环？

7 index 和 log 为何分开？

8 CLAUDE.md 为何是灵魂？

9 Obsidian 扮演什么角色？

10 和我已学的知识什么关系？

11 动手实验：迷你 wiki

12 坑与局限？

Q1 "第二大脑 / LLM-Wiki" 到底是什么？一句话说清

一句话：把你的文章、转录、PDF 丢进一个 Markdown 文件夹，让 Claude Code 这样的 LLM agent 去读、提炼、建页、互链、修护——你只负责喂料和提问，AI 负责一切簿记。

它有三个别名，指的是同一件事：**第二大脑 Second Brain**（个人知识管理圈叫法）、**LLM-Wiki**（Karpathy 在 gist 里的正式命名）、**活的 wiki**（强调它被持续维护、不会烂）。

为什么突然火？ 2026 年 4 月 Karpathy 把这个想法写成一篇 gist 《llm-wiki》，核心主张只有一句：与其让 AI 一次一次帮你写代码/回答问题，不如**用它构建一个持续增值的知识资产**。推文级传播号称千万浏览，社区一周内出现了多个 starter kit（本仓库克隆的 Aquinas-Protocol/second-brain 就是其中之一）。

类比：传统笔记 = 你自己养的花，浇水（维护）全靠自己，出差两周就枯；第二大脑 = 雇了一个不会累、不会忘、一次能整理 15 个房间的园丁，你只管往院子里扔种子（原始素材）。

思想源头其实很老：1945 年 Vannevar Bush 就设想过 **Memex**——一台私人知识机器，文档之间靠"联想轨迹"连接。Bush 当年没解决的问题是"谁来做维护"，LLM 恰好补上了这一块。

Q2 和 RAG 的本质区别：每次现检索 vs 增量编译

我们已在 [[RAG检索增强生成]] 学过：RAG 是"开卷考试"——查询时现场检索相关段落再作答。Karpathy 指出它的结构性缺陷：**LLM 在每个问题上都从零重新发现知识，没有任何积累**。要综合五份文档才能回答的微妙问题，每次都要重新找碎片、重新拼。

LLM-Wiki 反过来：**知识被"编译"一次、之后保持更新 (kept current) ，而不是每次查询重新推导 (re-derived)**。交叉引用早就建好了，矛盾早就被标出来了，综合早就反映了你读过的所有材料。

RAG：现检索

raw 文档堆 (静止)

问题1 → 检索碎片 → 答案 (丢弃)
问题2 → 再检索碎片 → 再答案 (丢弃)
每次都从零开始 🔄

价值不积累

范式切换

LLM-Wiki：增量编译

wiki (越养越肥)

新源 → LLM 读 → 更新 10~15 页
交叉引用✓ 矛盾标记✓ 综合✓
查询 = 读现成综合结果

每次都复利增值 📈

一个直白的对比：

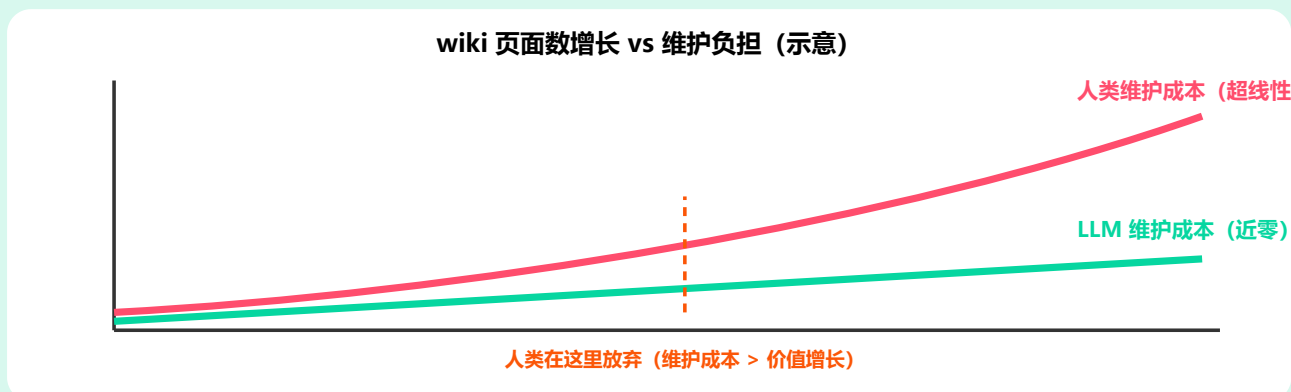
RAG 像"每次考试都临时翻书找答案"；LLM-Wiki 像"把书读一遍整理成自己的笔记本，之后越翻越快"。

关键差异不在检索技术，在"是否有一性性的编译投入 + 持续维护"。

注意二者不是敌人：Karpathy 也说 wiki 长大到几百页之后，可以配 **qmd** 这类本地混合检索 (BM25+向量，正是我们学过的 [[HNSW近似最近邻索引]] + 关键词检索的组合) 当外挂搜索引擎。

Q3 为什么人维护的 wiki 总会烂掉，换成 LLM 就行？

这是整个方法的经济学基础。Karpathy 的判断：“**维护知识库最累的不是阅读和思考，而是簿记 (bookkeeping)**”——更新交叉引用、保持摘要最新、标记新旧数据矛盾、跨几十个页面保持一致。



人类放弃 wiki 是因为**维护成本的增长快于价值的增长**。LLM 恰好补在短板上：不会厌倦、不会忘记更新引用、一次能改 15 个文件——**维护成本趋近于零，wiki 才第一次有可能“活着”**。

类比 (接 Q1 的花园)：花园荒掉从来不是因为种子不好，是因为浇水除草的活没人干。园丁便宜到忽略不计，花园自然就活了。

于是分工变成：**人的职责** = 策展素材、引导分析、提出好问题、思考意义；**LLM 的职责** = 其余的一切。

Q4 三层架构：raw / wiki / schema 怎么分权？为什么 raw 必须只读？



一句话记牢：人写 raw，LLM 写 wiki，两人一起写规则。

为什么 raw 只读？ 它是 **事实底座和审计线**：wiki 里任何一句综合结论，都能顺着 frontmatter 里的 sources：追溯到 raw 里的原文。raw 一旦可改，“知识被谁、依据什么改的”就说不清了——这正是我们学过的 [[JSONL事件日志与折叠模型]] 的同款思想：**只追加的事实层 + 可重建的派生层**。区别只是 wiki 不是从日志 fold 出来的，而是 LLM 从 raw 蒸馏出来的。

schema 层是 Karpathy 特意强调的：**CLAUDE.md (Codex 叫 AGENTS.md)** 是“让 LLM 成为守纪律的 wiki 维护者、而不是泛泛聊天机器人”的关键配置——没有它，每次会话 LLM 都要重新猜你的目录结构和写作规范。

Q5 原子笔记、双链、Zettelkasten: wiki 层的"语言"

LLM-Wiki 的 wiki 层不是新发明，它站在一百年笔记方法论的肩膀上：**Zettelkasten**（德语"卡片盒"）——社会学家卢曼用"一卡一想法 + 卡片互相引用"写了 70 多本书。三条规则：

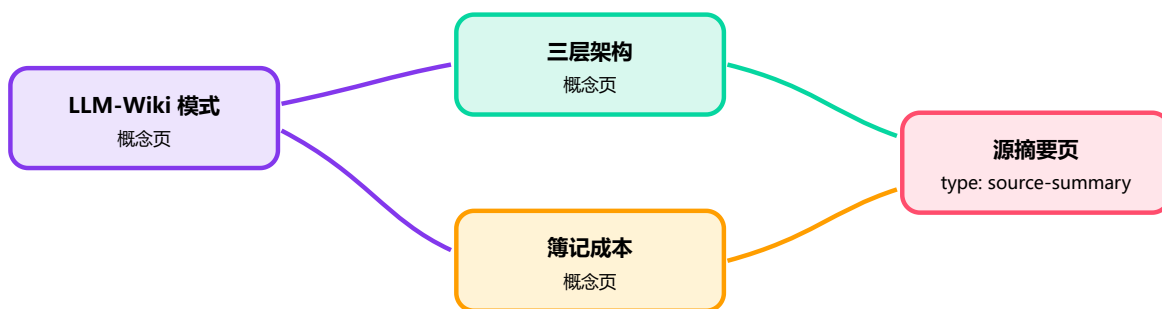
① 原子性：一页只讲一个概念

② 双链 [[页面名]]：页与页互相连接

③ 来源标注：每个论断指回 raw 原文

原子性为什么是硬要求？ 一页只讲一个概念，这一页才能被多个主题复用、被精确链接、被单独更新。一页塞五个概念，链接就只能指向"一堆东西"，知识图谱退化成一团浆糊。

原子页 + 双链 = 知识图谱（Obsidian graph view 看到的就是它）



每条 [[双链]] 都是双向的：你链接了谁，谁的"反向链接"面板自动出现你。

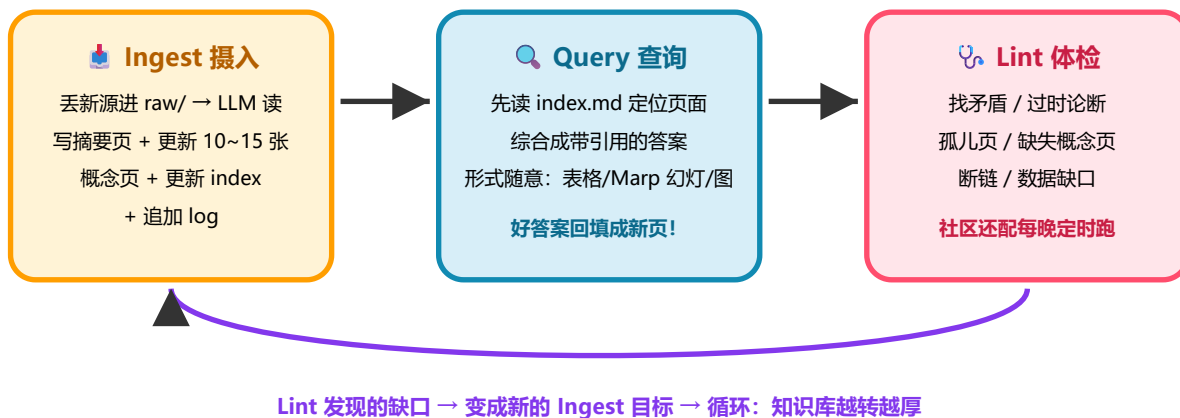
源摘要页（红） 汇聚所有概念页的引用 → 每个论断都能溯源到 raw。

页面还讲 frontmatter 纪律：YAML 头部写 title / type / sources / related / created 等字段——结构化元数据让检索和排序有据可依，也是 Obsidian **Dataview** 插件能自动生成动态表格的前提。

Q6 三大操作 Ingest / Query / Lint: 系统怎么循环运转?

Karpathy 把对 wiki 的一切操作收敛成三个动词，社区 starter kit 把它们做成了 Claude Code 的四条斜杠命令 (/ingest /query /log /lint) :

Query 的隐藏要点: 答案不能消失在聊天记录里, 要"归档回 wiki"——探索也复利。



两种节奏: Karpathy 本人偏好 **逐源精摄** (一次喂一篇, 读摘要、把关重点); 也可以 **批量夜摄** (starter kit 用 Windows 任务计划/每晚 3 点自动 ingest + lint, 睡一觉 wiki 自己长)。你完全可以先用前者建立信任, 再放开后者。

Q7 index.md 与 log.md：为什么拆成"目录"和"时间线"两个文件？

这是 gist 里最实用的工程细节。两个特殊文件职责正交：

wiki/index.md — 内容目录（面向"是什么"）	wiki/log.md — 时间线（面向"发生了什么"）
每页一行：[[链接]] + 一句话摘要，按类型分组	只追加：ingest/query/lint 每操作记一条，新的在上
每次 ingest 后重建	永不重写历史
查询时 LLM 先读它 定位相关页，再钻进去——在 ~100 源几百页规模，这一个文件就能替代 embedding RAG 基础设施	每条用统一前缀 ## [日期] 操作 标题，日志立刻变成 unix 工具可解析的流：grep 一下就是"最近 5 次操作"

```
grep "^## \[" wiki/log.md | tail -5
## [2026-10-02] ingest | LLM-Wiki 模式（示例原始素材一）（3 概念页）
## [2026-10-02] ingest | 原子笔记与 Zettelkasten（示例原始素材二）（3 概念页）
```

✅ **预期结果** 实验第 10 步实测：上面就是本机 mini-wiki 的真实 grep 输出——统一前缀让 log 免建数据库就能被检索。

记忆钩子：**index 回答"我有什么"**，**log 回答"我怎么变成今天这样的"**。前者给查询用，后者给审计和 LLM 的"最近上下文"用。

Q8 CLAUDE.md (schema) 为什么是整个系统的灵魂？

三层架构里最不起眼又最关键的是 schema。Karpathy 原话：“**这是关键配置文件——它让 LLM 成为守纪律的 wiki 维护者，而不是一个泛泛的聊天机器人**”。它写明：目录结构、页面 frontmatter 规范、链接语法、原子性要求、三种操作的具体流程。

用我们已经掌握的概念看，它一点也不神秘：

schema (CLAUDE.md) 是什么	已学过的同族概念
常驻上下文的“项目宪法”，每次会话自动生效	[[AgentSkills技能包]]：把方法论写成文件装给编程智能体
结构固定、槽位明确、可参数化（type 枚举、字段必填）	[[PromptAsCode提示词即代码]]：提示词协议化 → 可质检
规定“怎么做 ingest / lint”，让每次执行结果一致可查	[[质检Gate与自我纠错循环]]：lint 就是客观检查站
约束 LLM 只在 wiki/ 写、不碰 raw/	[[提示注入]] 防御里的最小授权思想

和 LLM 共同进化是 schema 的使用方式：发现 LLM 老犯某类错（比如忘了标注来源），就把对应规则加进 CLAUDE.md——规则库本身也是 wiki 的一部分，随使用越来越贴合你的领域。

Q9 Obsidian 在系统里扮演什么角色？没有它行不行？

Karpathy 那句流传最广的定位：**"Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase."**（Obsidian 是 IDE，LLM 是程序员，wiki 是代码库。）

注意这个定位里 Obsidian **不是必需品**——wiki 只是"一堆 Markdown 文件的 git 仓库"。Obsidian 提供的是三件免费好东西：

graph view：一眼看出哪些页是枢纽、哪些是孤儿

反链面板：每条双链的"谁链接了我"

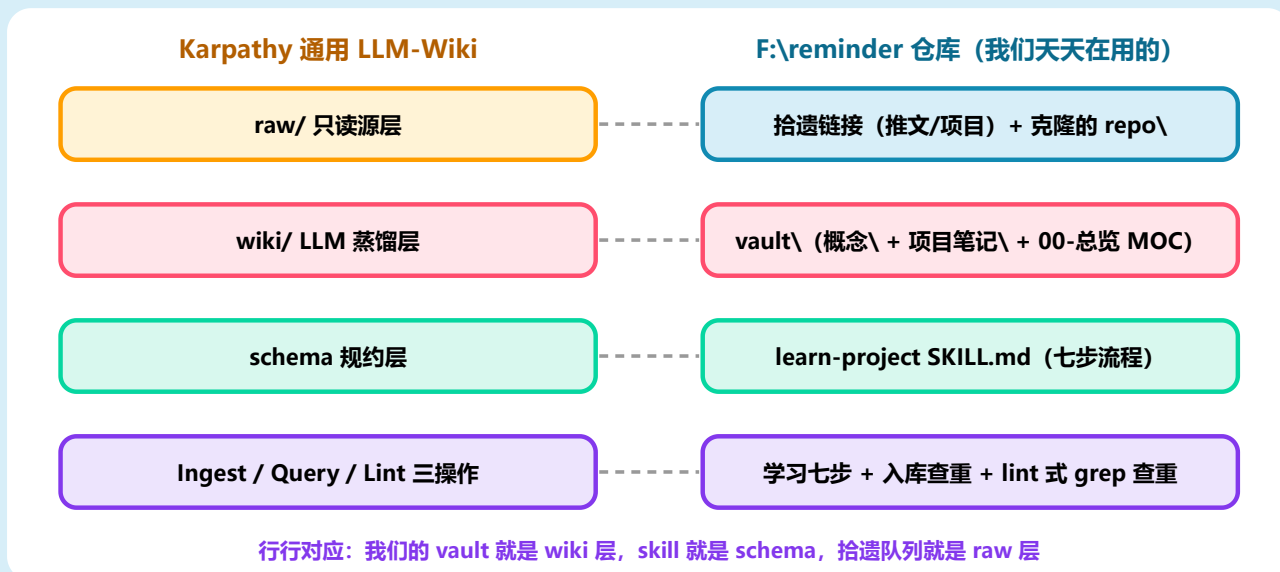
Web Clipper：网页一键转 Markdown 存入 raw/

这和我们学过的 [[本地优先与文件监听]] 完全同构：**数据主权在你手里的 .md 文件，应用只是文件的一个视图**。Obsidian 换成 VS Code、Logseq、甚至记事本，wiki 照样活着——agent (Claude Code) 读写的是文件系统，不依赖任何特定编辑器。附加好处：wiki 天然是个 git 仓库，版本历史、分支、协作全部白送。

可选增强（都来自 gist 的 tips）：图片本地化（附件目录固定到 raw/assets/，快捷键一键下载远端图）；**Marp** 用 wiki 内容直接生成幻灯片；**Dataview** 用 frontmatter 生成动态表格；wiki 长大后上 **qmd** 本地混合检索（CLI + MCP server 双形态）。

Q10和我已学的知识什么关系？（含：F:\reminder 本仓库就是它的实例）

最有意思的发现：F:\reminder 这个总仓库本身，就是一个带流水线的 LLM-Wiki 实例。逐层对上：



逐个挂接到已学概念：[[三层记忆]]——raw→wiki→schema 同样是“逐层压缩、可回溯”的记忆分层；

[[AgentSkills技能包]]——CLAUDE.md 和 SKILL.md 是同一物种；[[JSONL事件日志与折叠模型]]

——raw 只读 + log 只追加，同一“事实层不可变”哲学；

[[HNSW近似最近邻索引]]/[[RAG检索增强生成]]——index.md 目录式检索是它的“穷人版”，规模大了再用 qmd 补上。

Q11 动手实验：10 步搭一个会自检的迷你 LLM-Wiki（全部实测通过）

实验材料在 `F:\reminder\karpathy_second_brain\exercise\mini-wiki\`：一个 8 页的小 wiki + 三个零依赖 Python 工具（把 LLM 的簿记动作写成确定性脚本，让你看清每次 ingest 到底动了哪些文件）。所有命令已在 Windows 实机跑通，输出为真实记录。

第 1 步：建骨架 + schema

```
cd F:\reminder\karpathy_second_brain\exercise\mini-wiki
mkdir wiki\sources wiki\concepts
```

目的：立起 raw/wiki 双层 + CLAUDE.md 规约（已随材料提供，内容 = Q4/Q5 的规则）。实现了三层架构的空壳。

第 2 步：确认 raw/ 里有两份“剪藏”素材

```
dir raw
2026-10-02-llm-wiki-pattern.md  2026-10-02-atomic-notes.md
```

目的：模拟你往 raw/ 丢文章。raw 层从现在起只读。

第 3 步：ingest 前先体检（Lint 应发现空库）

```
python tools\wiki_lint.py
```

```
页面总数：0   双链总数：0   断链：0   孤儿页：0
发现 1 个问题：
  🚩 wiki/log.md 不存在（还没执行过任何操作？）
```

目的：先看“未处理状态”长什么样。验证：Lint 连“什么都没干”都能查出来。

第 4~5 步：Ingest 两份素材

```
python tools\mini_ingest.py raw\2026-10-02-llm-wiki-pattern.md
python tools\mini_ingest.py raw\2026-10-02-atomic-notes.md
```

```
[1/4] 源摘要页  wiki/sources/LLM-Wiki-模式-示例原始素材一.md
[2/4] 概念页    wiki/concepts/模式是什么.md      (+2 张)
[3/4] 索引重建  wiki/index.md (共 4 页)
[4/4] 日志追加  wiki/log.md
```

目的：跑一次完整 Ingest。一次 ingest = 1 源摘要页 + 3 原子概念页 + index 重建 + log 追加——Karpathy 说"一个源触及 10~15 页"，小例子 5 页同理。

✅ **预期结果** wiki/sources 出现 2 页、wiki/concepts 出现 6 页、index.md 8 行条目、log.md 2 条记录。

第 6 步：ingest 后再体检（应全绿）

```
python tools\wiki_lint.py
```

页面总数：8 双链总数：24 断链：0 孤儿页：0

✅ 全部通过：frontmatter / 断链 / 孤儿页 / index 覆盖 / log 格式 / sources 路径

第 7 步：故意埋雷，验证 Lint 真的能抓

新建一页 断链演示页.md，内容链接到 [[不存在的页]]，再跑 lint：

断链：1 孤儿页：1

发现 3 个问题：

- ❌ concepts\断链演示页.md：断链 [[不存在的页]]（目标页不存在）
- ⚠️ 孤儿页：断链演示页（没有任何页面链接它）
- ❌ index.md 未收录页面：断链演示页

删除该页后复跑 lint → 恢复 ✅。看到 lint 抓到真问题，才证明体检不是摆设。

第 8 步：Query——先读索引，再给页面排名

```
python tools\wiki_query.py wiki
python tools\wiki_query.py 原子
```

[Query] 先读索引 wiki\index.md ...

索引命中：- [[为什么手工-wiki-会烂掉]] - ...

[wiki] 检索结果（8 页命中）：

- 14.5 sources\LLM-Wiki-模式-示例原始素材一.md （入链 3）
- 13.5 concepts\为什么手工-wiki-会烂掉.md （入链 3）

目的：体验"index 优先 + 标题加权 x3 / 元数据 x2 / 正文 x1 / 入链 x0.5"的目录式检索。验证：与"wiki"最相关的页排在最前。

第 9 步：验证 log 是 grep 可解析时间线

```
grep "^## \" wiki\log.md
```

[2026-10-02] ingest | LLM-Wiki 模式（示例原始素材一）（3 概念页）

[2026-10-02] ingest | 原子笔记与 Zettelkasten（示例原始素材二）（3 概念页）

第 10 步：用 Obsidian（或任意 Markdown 编辑器）打开 mini-wiki

把 mini-wiki 文件夹作为 vault 打开 → 看 graph view 的连线、点开任一页看反链面板。**这一步就是"Obsidian 是 IDE"的体感时刻。**

升级路线：把 mini-wiki 里的确定性 ingest 换成真的 Claude Code + 一份 CLAUDE.md，就是完整版第二大脑；再配 starter kit 的四条斜杠命令和夜间定时任务，就是社区生产级形态。

Q12 坑与局限：上手前必须知道的四件事

① **幻觉污染。**wiki 层由 LLM 写——它编造的内容一旦进了 wiki，就被"编译"成了看似可信的知识，还会被后续 ingest 引用放大。对策：**每个论断强制标 sources 回指 raw**（原子性三规则的第三条），lint 时抽查溯源。

② **隐私。**raw/ 里常塞对话导出、录音转录、私人笔记——虽然全流程本地（这正是选 Claude Code CLI 而非网页版的原因），但 wiki 若推到公开 git 远程就泄露了。starter kit 甚至开篇先讲隐私：敏感素材留在本地、仓库设私有。

③ **成本与规模。**一次 ingest 动辄十几页的读+写，token 消耗可观；几百页以内 index.md 目录检索够用，再大就要上 qmd 混合检索、甚至重新引入向量库（[[向量数据库]] 那套并不白学，只是"不到规模不用上"）。

④ **编译错误的累积。**RAG 的错误随查询结束消散；wiki 的错误会**驻留并复利**。所以 Lint 不是可选项而是安全机制——就像代码要跑测试，wiki 要定期体检。人的"策展"职责里，审 LLM 的摘要质量排第一位。

一句话总结

第二大脑 = raw（人写，只读）+ wiki（LLM 写，原子页+双链）+ schema（共同进化）三件套，靠 Ingest/Query/Lint 三操作转起来；它成立的经济学前提只有一个——LLM 把维护成本打到近零。



这套方法给你什么（能力清单）

- ① **把"收藏夹吃灰"变成知识资产**：文章/视频转录/PDF 丢进 raw/，让 agent 吃进去——读过的东西第一次开始复利。
- ② **读书伴生 wiki**：每读一章 ingest 一次，人物页/主题页自动生长，读完书 wiki 也成型（Tolkien Gateway 那种 fan wiki 一个人就能养）。
- ③ **科研/调研渐进综合**：几个月深挖一个课题，论断带溯源、矛盾被标记、综述页始终反映最新阅读。
- ④ **团队知识库无人值守维护**：会议转录、Slack 导出夜间自动 ingest，wiki 不再依赖"那个愿意整理的人"。
- ⑤ **直接用于本仓库**：F:\reminder 的 vault 已经是实例——本指南的 Q10 映射表可以直接拿来优化 learn-project 流程（比如给 vault 补一个 lint 脚本）。
- ⑥ **问答成果回填**：以后每次问 vault 得到的好答案，按 Q6 的"归档回 wiki"原则存成新页，探索本身也复利。